

Change Notification: Allocations based Scheduling

Last Update: Aug 19, 2026

Status: enforcement is live on Holmes as of 07/31/2026. All other clusters, including Jupiter, are still on the previous policy.

This document explains the allocation hierarchy (organizations, budgets, workspace groups, and per-scope runtime quotas / slot limits) and the reporting, UI, and scheduling built on it. The job-level minimum-runtime model it builds on is defined in full in the companion Change Proposal: Minimum Runtimes; because the two are so tightly coupled, its key points are recapped in Minimum Runtimes (Recap) below.

Where to start

To do this	See
Understand what changed on Holmes on 07/31	What Is Live On Holmes
Set an allocation for a workspace group or workspace	How To Set Allocations
Pull usage numbers and compare them to a target allocation	Reviewing Usage Against Allocations
Work out which slot limit is actually capping your jobs	Which Limit Applies Where
Decide how to configure your team	Recommended Configuration
Understand why a job was preempted	How The Scheduler Orders Work

Objective

Give organizations a declarative, hierarchical model for dividing a cluster's GPU time among teams, and give managers the tools to administer it. This is the accounting and delegation layer of the budget-based scheduler effort: it establishes who is entitled to what share of a cluster (runtime quotas) and how many GPUs a scope may hold at once (slot limits), across a tree of Organization -> Budget -> Workspace Group -> Workspace.

Motivation

Beaker's original controls (priority + preemptibility, now min-runtime) govern a single job's fate, but they say nothing about fair division of a cluster across teams over time. Budgets were only billing labels; there was no way for a manager to say "Team A gets 40% of this cluster, and no more than 32 concurrent GPUs." The allocation hierarchy adds that:

- **Allocation percentage** — each node is given a share of its parent's allocation (in the UI, the "Target Relative (vs. Parent)" you set). A team's actual share of the whole cluster is the product of the percentages down its path — shown as "Target Allocation (% of Cluster)" — e.g. Budget 40% x Group 60% x Workspace 50% = 12% of the cluster.
- **Slot limit** — each node may set an optional slot limit: the maximum number of GPUs it may use at once. When a limit is set at more than one level, the smallest one along the path wins. Slot limits apply to allocated jobs only; see Which Limit Applies Where.
- **Delegation** — budget managers and workspace-group managers can administer allocations for their subtree without org-admin involvement.

The scheduler change these controls support also addresses priority inflation. Under the previous policy every user is incentivized to use the highest priority available to them and to mark work non-preemptible, which leads to an outcome misaligned with organizational goals. Under the new policy, allocated is the new urgent: where people previously needed urgent priority, they now need an allocation, and priority becomes a tool for ordering work within a team rather than against the whole organization.

Allocation Related Concepts

Allocation hierarchy Organization -> Budget -> Workspace Group -> Workspace. Each allocation applies to one entity in the tree on a specific cluster, and sets that entity's allocation percentage (its share of the parent) and an optional slot limit (the maximum GPUs it may use at once).

```
None
Organization: ai2
|
+-- Budget: oe-other
|   |
|   +-- Workspace Group: oe-scaling      (allocation %, slot limit 304)
|   |   +-- Workspace: olmo3            (allocation %, optional slot limit)
|   |   +-- Workspace: oe-training      (allocation %, optional slot limit)
|   |
|   +-- Workspace Group: oe-encoder      (allocation %, optional slot limit)
|       +-- Workspace: oe-encoder-dev
```

```
|
+-- Budget: oe-omai
    |
    +-- Workspace Group: omai-agents
        +-- Workspace: omai-multimodal
```

Workspace groups: an intermediate allocation node that bundles several workspaces under one budget. It helps divide a cluster's GPU time across working teams, and delegate who gets to administer that division. A workspace can be placed into a group; once it is, it inherits the group's budget and cannot override it (a group belongs to exactly one budget). The workspace group is also the level at which the scheduler does its primary balancing.

Budget & Workspace-group managers: an org member granted authority to administer a specific allocation node's subtree — without needing org-admin rights. Managers are the delegation mechanism that makes the hierarchy scale: the org admin doesn't have to configure every team's internal split; each level manages itself. (See UI: Org -> Permissions tab below.)

Minimum Runtimes (Recap)

See Change Proposal: Minimum Runtimes for the full details.

Two settings replaced the old preemptible switch. A workload now declares a minimum runtime (how long its jobs are protected from interruption; default none) and whether it auto-resumes (whether an interrupted job is automatically restarted). The old preemptible on/off flag is deprecated and mapped to these automatically.

This defines "allocated" vs. "unallocated." A job whose minimum runtime is greater than zero is allocated; a job with no minimum runtime is unallocated. This is the same split the allocation hierarchy, usage reports, and slot limits throughout this document rely on.

Protection window. An allocated job cannot be interrupted — not even by system automation (e.g. draining an unhealthy node) — until its minimum runtime elapses; after that it becomes interruptible by higher-priority work. Unallocated jobs are always interruptible. If an auto-resuming job is interrupted, its replacement starts with a fresh protection window.

Bounds. Minimum runtimes are capped to a sensible range (roughly 5 minutes to 8 hours), so protection can't be set arbitrarily long. Having small minimum runtimes is essential to both properly balance allocations (time slice to match the configured desired throughput) and to conduct timely operational repairs.

Two slot-limit lanes. Allocated jobs count against a workspace's non-preemptible slot limit; unallocated jobs against its preemptible slot limit.

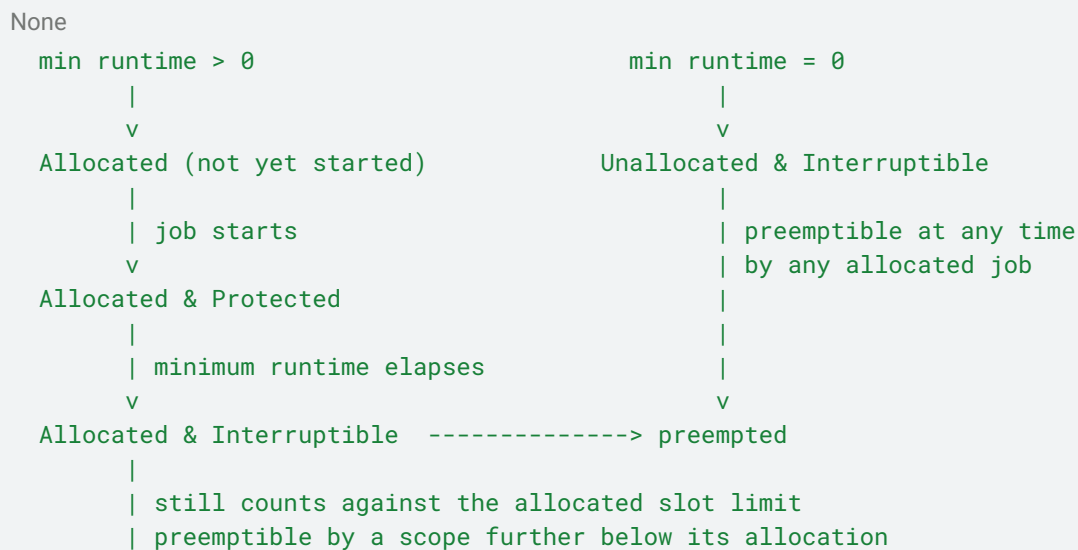
Turned on per cluster. Minimum-runtime enforcement rolls out one cluster at a time. On a cluster that hasn't migrated, allocated jobs stay protected indefinitely (today's behavior); on a migrated cluster, protection lasts only for the minimum runtime, and workloads that were previously marked non-preemptible are treated as having the maximum (8-hour) minimum runtime.

An allocated job stays allocated after its protection expires

When an allocated job passes its minimum runtime it becomes Allocated & Interruptible. It does not become unallocated, and it continues to occupy a slot against the allocated slot limit for its workspace and workspace group.

This is deliberate. If jobs became unallocated when their protection lapsed, a team's allocated slot count would free up continuously, allowing them to launch further allocated jobs and interrupt other workspace groups' work. Keeping the job allocated closes that loophole.

The job states, as shown in the per-job badge:



Rollout Status By Cluster

Enforcement rolls out one cluster at a time, and teams are notified before it takes effect on a cluster.

Behavior	Holmes (new policy, live 07/31)	Jupiter and others (previous policy)
Allocation required to run allocated jobs	Yes	No
Allocation slot limits enforced	Yes, at budget / group / workspace, per cluster	No; recorded and displayed only
Allocation percentages affect scheduling	Yes, balanced over a rolling window	No; informational only
Protection after minimum runtime expires	Ends; job becomes interruptible	Allocated jobs stay protected indefinitely
Old per-workspace global slot limits	Still applied, in addition to allocation limits	The only limits that apply
Unallocated jobs	Backfill only; always behind allocated work	Behave as before

To check a cluster's live policy:

```
None
beaker cluster scheduler-policy get ai2/holmes
```

What Has Been Rolled Out

Everything in this section has been rolled out in the API, CLI, and UI as of 07/20/2026.

Allocation management & reporting tools

UI: Cluster -> Allocations tab (ex:

<https://beaker.org/orgs/ai2/clusters/jupiter/allocations?expanded=oe-omai%2Fomai-agents>) By cluster allocation hierarchical table (Budget -> Group -> Workspace) showing GPU usage hours (allocated vs. unallocated), allocated runtime quota %, and slot limits. A drop-down menu in each row also brings about an edit view ([.../allocations/edit](#)) that allows users with permissions to set allocations for org/budget/workspace-group parents.

UI: Org -> Permissions tab (<https://beaker.org/orgs/ai2/permissions>) By organization table for hierarchical budget / workspace-group / workspace management: create/edit/rename/deprecate budgets; create/edit/remove workspace groups; add/remove workspaces to groups; manager editing.

UI: Workspace -> Permissions tab (ex:

<https://beaker.org/orgs/ai2/workspaces/jjyang-ds/permissions>)

UI: Workspace -> Allocations tab (ex:

<https://beaker.org/orgs/ai2/workspaces/oe-agents/allocations>) By cluster allocations for a workspace.

CLI: workspace-group `beaker workspace-group ... , beaker workspace-group managers ... , beaker workspace set-group.`

CLI: allocation read/set `beaker allocation list` and `beaker allocation set-child` (dry-run by default, `--apply`, `--reason`, spec-file driven).

Allocated-vs-unallocated job visibility (UI for Minimum Runtime Changes)

The classification itself is defined in the Minimum Runtimes proposal; what shipped here is the surfacing of it to users:

- A per-job protection-state badge shown in running-jobs tables, the cluster queue, node job lists, and the workload timeline, with four states: Unallocated & Interruptible, Allocated (not yet started), Allocated & Protected (with a countdown to when the minimum runtime elapses, or "protected indefinitely" on non-min-runtime clusters), and Allocated & Interruptible.
- A workload's interruption behavior is now described by its minimum runtime and whether it auto-resumes, instead of the old single preemptible on/off switch. The preemptible setting is deprecated: you can still supply it when submitting a workload (it's mapped to the new settings automatically), but it no longer appears as a property of a job — the UI, CLI, and reports describe jobs in terms of minimum runtime / auto-resume and the allocated-vs-unallocated status above.
- Each workspace still has its own two slot limits, now labeled "Allocated Slot Limit" and "Unallocated Slot Limit". These per-workspace limits are still shown in the UI and still apply. They are slated to be replaced by the allocation-based slot limits once those are enforced on all clusters.

What Is Live On Holmes

The new scheduler policy went live on Holmes on 07/31/2026. Three changes:

1. Having an allocation is now required to run allocated jobs (min runtime > 0). If a workload sets a minimum runtime greater than zero and there is no covering allocation for its workspace on Holmes, it will not schedule. Any workspace that wants to use Holmes should be

in a workspace group with an allocation and have an allocation itself. The current list is at <https://beaker.org/orgs/ai2/clusters/holmes/allocations>.

2. Allocation slot limits are now enforced. Budget-, group-, and workspace-level per-cluster limits cap concurrent GPUs. If they trigger you'll see a failed scheduling message with details.

3. The queue order has changed.

How The Scheduler Orders Work

None

```
Job enters the queue
|
+--> immediate priority?
|   Yes -> front of the queue. Interrupts any job, including
|           protected ones. Ordered by age. Org admins only.
|
+--> allocated (min runtime > 0)?
|   Yes -> sorted by, in order:
|       1. workspace group allocation balance
|           (groups farther from their allotted usage go first)
|       2. workspace allocation balance
|       3. priority
|       4. workload age
|
+--> unallocated (min runtime = 0)?
|   Yes -> used for backfill, always behind every allocated job.
|           Sorted by priority, then workload age.
```

Consequences of this ordering:

- Priority no longer competes globally. Because workspace group and workspace balance are evaluated first, your priority cannot place your job ahead of those in other workspaces. Priority orders work within your own scope. It will not prevent preemption if you are running ahead of your allocated throughput and other workload submitters are running behind theirs.
- Only allocated usage counts towards an allocation's balance. Unallocated hours do not make a scope look over its share.
- Any time a job is preempted for balance reasons, it is because there are other workloads waiting which have received less of their allocation. (Note that priority-based preemption can also occur within the same workspace, regardless of balance.)
- Immediate priority remains an escape hatch, but will interrupt any protected jobs and is reserved for org admins only.

Note that after some discussion the policy for Holmes was changed to sort workspace group -> workspace -> priority -> age rather than workspace group -> priority -> workspace -> age.

The usage window

Balance is measured over a rolling window rather than instantaneously. Current Holmes settings:

Setting	Value	What it controls
lookback	120h (5 days)	How far back usage is considered
usageDecayHalfLife	120h	How quickly older usage stops counting against a scope
forgivenessHorizon	24h	Grace period before recent usage counts against a scope (exact semantics to be confirmed)

An allocation is therefore a guarantee over a period of time, not a point-in-time guarantee: e.g. this workspace gets 10k hours over N days. If a team needs a point-in-time guarantee, use slot limits as described in the next section.

Two Applications Of Allocations

Runtime percentages and slot limits produce two different behaviors. Both are supported; the choice should be deliberate.

Exclusive access

Set slot limits so that the limits across sibling scopes sum to at most the cluster's capacity. Each group is given a subset of the cluster and can run immediately if it is not competing with itself.

None

Allocations:

Budget A

4 GPU Limit

50% runtime

+-----+

| GPU 1 GPU 2 |

| GPU 5 GPU 6 |

Budget B

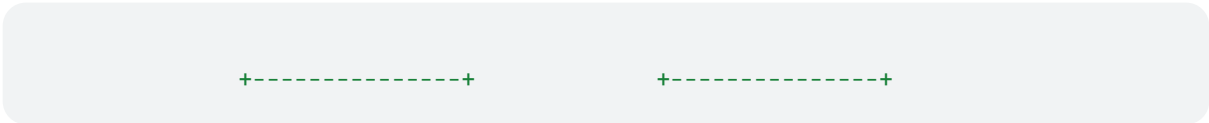
4 GPU Limit

50% runtime

+-----+

| GPU 3 GPU 4 |

| GPU 7 GPU 8 |



The configuration for this is:

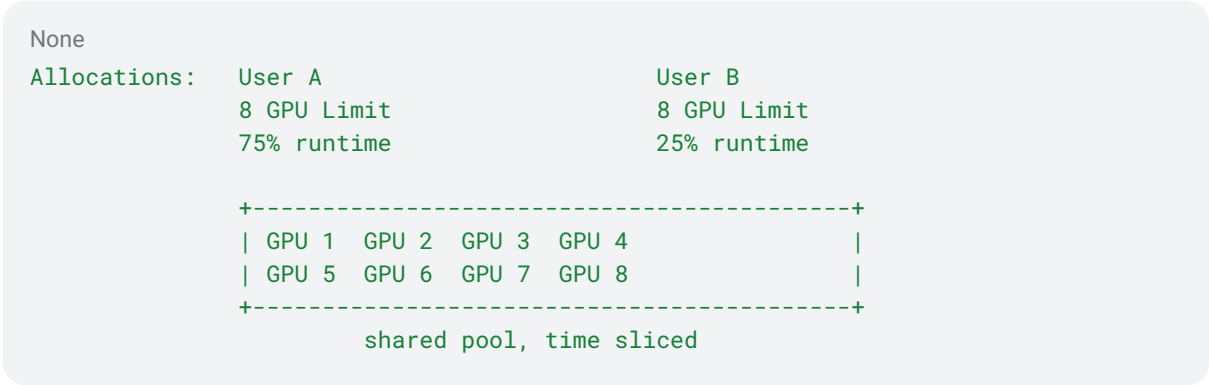
- Total number of allocated slots across workspace groups == total number of GPUs on the cluster
- Each workspace group's cumulative allocated slot count == their budget (where all budgets add up to the total number of GPUs on the cluster)

Adding slot limits to the workspaces inside a group extends exclusivity down a level, giving each workspace its own subset. Omitting them means any individual workspace can use the whole parent group's slots at once, and the scheduler will attempt to align usage to the percentage over the rolling window instead (e.g. oe-encoder = 7.9% of the cluster over a week = 7k hours).

This configuration can be undesirable, as demand shifts across budgets, workspaces, and users over time, and constraining concurrency comes at the cost of total utilization.

Interleaved execution

Leave slot limits loose or unset and let the runtime percentages do the work. All scopes share the pool, and minimum runtimes provide the time slicing that brings usage in line with the target percentages over the window.



	Exclusive access	Interleaved execution
Configured by	Slot limits summing to capacity	Percentages, with loose or no slot limits
Guarantee	Point-in-time GPU count	GPU hours over the window

	Exclusive access	Interleaved execution
Can burst above share	No	Yes, when others are idle
Utilization	Lower	Higher
Suited to	Long pretraining runs, firm commitments	Bursty experimentation, many smaller jobs

Which Limit Applies Where

There are currently two slot-limit systems, and on Holmes both apply at the same time. A job can be capped by either.

Limit	Where it is set	Scope	Applies to	Enforced on
Allocation slot limit (new)	Cluster -> Allocations tab, at budget / group / workspace	Per cluster, hierarchical	Allocated jobs only	Holmes
Global workspace allocated slot limit (old)	Workspace page	All clusters, single workspace	Allocated jobs	All clusters
Global workspace unallocated slot limit (old)	Workspace page	All clusters, single workspace	Unallocated jobs	All clusters

An allocated job on Holmes counts toward both its per-cluster allocation limits (up the whole hierarchy) and the workspace's old global allocated limit.

Note that the "Slot Limit" column in the Allocations tab governs allocated GPU slots, not all slots. The tooltip states this ("Maximum number of allocated GPU slots this entity may use simultaneously") but the column header does not yet. Adding up urgent slots in the dashboard pie charts and comparing them to a group's slot limit will not reconcile, because much of that urgent usage may be unallocated, which the limit does not govern. A rename of the column header to "Allocated job slot limit" is planned.

How To Set Allocations

Prerequisites

- To administer a subtree you must be an org admin, or a manager of the budget or workspace group in question.
- A workspace must be in a workspace group to inherit a budget-level allocation. Grouped workspaces inherit the group's budget and cannot override it.

In the UI

Task	Where
Create/edit/rename/deprecate budgets; create/edit/remove workspace groups; add/remove workspaces to groups; manager editing	Org -> Permissions, https://beaker.org/orgs/ai2/permissions
View the hierarchy for a cluster and set percentages and slot limits	Cluster -> Allocations, https://beaker.org/orgs/ai2/clusters//allocations , row drop-down -> Edit
See one workspace's allocations across clusters	Workspace -> Allocations, https://beaker.org/orgs/ai2/workspaces//allocations
Set a workspace's old global slot limits and max priority	Workspace -> Permissions, https://beaker.org/orgs/ai2/workspaces//permissions

The Cluster -> Allocations table is the main working surface. Rows can be expanded by scope, e.g. <https://beaker.org/orgs/ai2/clusters/holmes/allocations?expanded=oe-other%2Foe-scaling>.

From the CLI

```
None
# Workspace groups
beaker workspace-group ...           # create / list / edit groups
beaker workspace-group managers ...  # delegate administration
beaker workspace set-group ...       # move a workspace into a group

# Allocations
beaker allocation list                # read current allocations
beaker allocation set-child           # set a child's percentage and slot limit
                                     # dry-run by default
                                     # --apply    commit the change
                                     # --reason   recorded with the change
                                     # spec-file  driven for bulk edits
```

`beaker allocation set-child` is dry-run by default: run it once to review the change, then re-run with `--apply`. Every change is versioned, which also keeps two people from unknowingly overwriting each other's edits.

Example: capping a workspace group

To ensure the oe-scaling workspace group does not exceed 304 allocated slots on Holmes:

1. Set the slot limit on the group's Holmes allocation to 304. Any jobs laddering up to that workspace group will always be capped at that limit.
2. Setting limits on the individual workspaces inside the group is optional, and they do not need to add up to 304. Adding them provides exclusive access for each workspace. Omitting them means any individual workspace can use the whole 304 parent group slots at once, with the scheduler aligning usage to the percentage over the window.
3. You do not need to update the workspaces globally for the Holmes cap to apply. Those workspaces may still be limited by the global limits, since both limits are applied.

Reviewing Usage Against Allocations

Where the numbers are today:

- **Cluster -> Allocations:** the hierarchical table showing GPU usage hours split allocated vs. unallocated, alongside each node's target allocation % and slot limit. This is the primary view for comparing consumption against a target share.
- **Workspace -> Allocations:** the same view from a single workspace's perspective, across clusters.
- **Cluster dashboard:** pie charts of current usage. Useful for what is running now, but not for reconciling against slot limits, since the charts combine allocated and unallocated usage.
- **CLI:** `beaker allocation list` for scripted reads of the current configuration.

To convert a target percentage into expected hours:

None

```
expected GPU hours = cluster GPUs x 24 x days x target allocation % of cluster
```

For example, a workspace group at 7.9% of the cluster over a week is roughly 7k GPU hours. Compare that against the allocated hours shown for the group in the Allocations tab. A scope well under its target will be favored by the scheduler; a scope over its target will yield to others.

Points to keep in mind when reading these numbers:

1. Only allocated usage counts towards an allocation's balance. Unallocated hours appear in reports but do not affect scheduling order.

2. GPU-usage reports group time by a workload's budget and workspace group. Those are read from the workspace's current settings, so if a workspace changes budget or moves between groups, its older usage may not line up perfectly with where it sits today. More precise, point-in-time attribution is being rolled in, but until then treat historical usage-by-budget / by-group as approximate around any recent reorganizations.
3. There is no consumption-to-date chart yet. Recent usage graphs are being added to the UI, since recent usage is the primary decider of which allocations are prioritized. Until then, the Allocations tab plus the calculation above is the available method.
4. There is no allocation change history in the UI yet. Every change is recorded and versioned, but there is no timeline view to browse.

Recommended Configuration

While there are both types of clusters (new allocation policy vs. previous policy) and both types of slot limits (new hierarchical per-cluster allocation limits that only limit allocated jobs, and old global workspace limits for either unallocated or allocated jobs), the guidance is:

1. Across all work, guide folks toward using allocated jobs (min runtime > 0). On Jupiter, this should have no effect for urgent users, and will slightly reduce how often non-urgent work can be preempted. On Holmes, running allocated is essential to actually getting the compute that allocations guarantee on a best-effort basis; unallocated work on Holmes should be thought of as best effort / filler. The main cost of switching is coordination with whoever manages the old workspace-level slot limits, since the allocated and unallocated values will likely need swapping.
2. For any workspaces that want to use Holmes, make sure they're in a workspace group with an allocation and have an allocation themselves. The optional per-cluster allocation slot limit will only affect allocated jobs on Holmes, while the global allocated workspace limit will continue to limit allocated jobs on both Holmes and all other clusters (in addition to the per-cluster limit); allocated jobs on Holmes will count toward both limits.
3. Where possible, set the old global unallocated workspace limit to 0 and use only the global allocated limit.
4. Use group-level slot limits as the real cap, and decide whether to add workspace-level limits (exclusive access) or not (interleaved execution).

On switching to allocated jobs on Jupiter: if you're running unallocated jobs today on Jupiter and switch to allocated, your guarantee only increases. Today on Jupiter an urgent job won't preempt another urgent job, regardless of allocation status, even if the running job has exceeded its min runtime (without allocations, there's no way to balance overall usage so there's no benefit to that preemption). And on Jupiter, if you're not running at urgent and happen to land an allocated job, you have the benefit of getting your min runtime. So there is no downside to switching to allocated on Jupiter aside from fiddling with slot limits.

Known Discrepancies & Gotchas

Items 1 and 2 are resolved on Holmes but still apply on every other cluster.

1. Allocation slot limits aren't enforced off Holmes — so there are two places to set limits

You can set a slot limit on a cluster's allocation — at the budget, group, or workspace level — in the Allocations tab, and the UI will show it and roll it up through the hierarchy. On a cluster that has not migrated it has no effect on scheduling. The only slot limit that actually caps concurrent GPUs there is the older per-workspace control on a workspace's own page. The exact symptom to expect: you set a slot limit for a cluster / budget / group, but a workspace in that scope keeps running up to the limit configured on the workspace itself.

Interim guidance: on clusters other than Holmes, keep using the per-workspace control to actually cap usage, and treat the allocation-level limits as planning ahead for when the scheduler starts honoring them.

2. Allocation percentages are informational off Holmes

The allocation percentages (and the effective share shown in the Allocations tables) do not change scheduling order or which jobs get interrupted on clusters that have not migrated. They describe the intended division of the cluster; a team running over its share is not throttled there.

3. Slot limits count allocated jobs only, but the column header does not say so

See Which Limit Applies Where. A column header rename is planned.

4. Usage reports attribute GPU time by current settings

GPU-usage reports group time by a workload's budget and workspace group. Those are read from the workspace's current settings, so if a workspace changes budget or moves between groups, its older usage may not line up perfectly with where it sits today.

5. Queued jobs can only be cancelled by their owner

Workspace admins currently see only a delete option, which also removes previous attempts and logs. If a job is blocking a cluster and its owner is unavailable, escalate in #beaker-users. A cancel action for workspace admins is being tracked.

6. The 8 hour minimum runtime ceiling does not cover multi-week runs

A pretraining run expected to take two to three weeks cannot be protected end to end by minimum runtime alone. The intended mechanism for this is exclusive access through slot limits rather than longer protection windows; immediate priority exists as an org-admin escape hatch. Such workloads should be auto-resuming. Allocated jobs should be resumable, and if they're not this is a bug that should be surfaced.

What Is Still Pending

Scheduler rollout

- Enforcement on the remaining clusters, turned on one cluster at a time, with teams notified before it takes effect.

Better waiting-job feedback

- Estimated start times for waiting allocated jobs, updated until the job starts.
- Clearer "why is my job waiting?" messaging in the UI — explaining which limit, quota, or recent usage is holding a job back — so users aren't left guessing, including which jobs are ahead in the queue.

UI clean-up

- Renaming the slot limit column to reflect that it governs allocated jobs.
- Changes to the usage charts so allocated and unallocated usage are distinguishable.
- An expanded policy section in the UI documenting the sort order in effect on each cluster.
- Recent usage graphs, since recent usage is the primary decider of which allocations are prioritized.
- An allocation change history view.
- Retiring the old per-workspace slot-limit control once allocation slot limits are the ones that actually take effect, so there is a single, consistent place to set limits.
- Tighter default sharing for new workspaces (new workspaces defaulting to read-only for the broader org rather than open write access), as part of tightening things up at Ai2's current scale.
- A cancel action for queued jobs available to workspace admins.

FAQ

Q: If a workspace is in a group, which budget do its workloads use? The group's budget — grouped workspaces inherit it and can't override it.

Q: What does "effective" allocation mean? It's the actual cap that applies to a node once it's combined with its ancestors. Two rules, with an example:

- Runtime quota multiplies down the chain. If Budget A has 40% of the cluster, Group X has 60% of A, and Workspace a has 50% of X, then Workspace a's effective share is $40\% \times 60\% \times 50\% = 12\%$ of the cluster (shown as "Target Allocation (% of Cluster)").
- Slot limit takes the smallest along the chain. If Budget A allows 64 slots, Group X 32, and Workspace a 40, then Workspace a's effective slot limit is $\min(64, 32, 40) = 32$ — a child can never exceed an ancestor's cap, even when its own number is higher.

Q: Does an allocation guarantee my team N GPUs at any point in time? Not on its own. It's not a point-in-time guarantee, it's a guarantee over a period of time, e.g. this workspace gets 10k hours over N days. The slot limits in the allocation system can provide exclusive access, but they're an optional setting to complement the runtime tracking mechanism.

Q: Does a job become unallocated after its minimum runtime elapses, freeing up the allocated slot count for that workspace or workspace group? No. It remains allocated even after it becomes interruptible, and it keeps using the allocated slot.

Q: Once a job becomes interruptible, who can interrupt it? An allocated job from a scope that has used less of its allocation, but only if the victim's scope has used more than its allocation. A scope below its share is never a valid victim on balance grounds.

Q: I set a slot limit on a budget allocation but my team's jobs aren't being limited by it. Why? On Holmes, check whether the usage being measured is actually allocated; the allocated limits only apply to allocated jobs, and urgent-but-unallocated jobs do not count against them. On other clusters, allocation slot limits are not enforced yet and the scheduler honors only the slot limit set directly on the workspace.

Q: Do the workspace slot limits inside a group need to add up to the group's limit? No. Any jobs laddering up to that workspace group will always be capped at the group's limit. Adding workspace-level limits provides exclusive access for each workspace; omitting them lets any individual workspace use the whole group limit at once.

Q: Can I prioritize my critical path jobs above my other jobs? Within a workspace group, yes: priority is the third sort key, after workspace group and workspace allocation balance. The managers of the workspace group also have the ability to set the max priority of the workspaces in that group. If all such workspaces allow the same max (e.g. urgent) the scheduler falls back to workspace level allocation balancing; if they allow different priorities, this allows an explicit bypass of the workspace level allocations within that group. Priority will not place a job ahead of work in another workspace group.

Q: Can I see the history of allocation changes? Not through the UI yet. Every change is recorded — who made it, when, and the reason — and each change is versioned (which also keeps two people from unknowingly overwriting each other's edits). But there is no

history/timeline view to browse past changes today; surfacing that record in the UI is planned for future work.

Q: Will my current workspace slot limits stop working when enforcement ships? Not right away. Enforcement rolls out per cluster (like the min-runtime policy). Once allocations are enforced on all clusters, the per-workspace slot limits will be disabled in favor of the allocation-based limits — but not before, so nothing breaks mid-rollout. On Holmes today, both limits apply.

Q: How do I check which policy a cluster is running? `beaker cluster scheduler-policy get ai2/<cluster>`.

Appendix: Holmes scheduler policy as of 07/31/2026

Retrieved with `beaker cluster scheduler-policy get ai2/holmes:`

```
None
backfill: unallocatedOnly
interruptibleAfterMinRuntime: true
enforceAllocationSlotLimits: true
requireCoveringAllocation: true
maxSessionTimeout: 8h0m0s
minRuntimeLowerBound: 5m0s
minRuntimeUpperBound: 8h0m0s
usageWindow:
  lookback: 120h0m0s
  usageDecayHalfLife: 120h0m0s
  forgivenessHorizon: 24h0m0s
sort:
  partitions:
    - match:
        kind: priority
        priority: immediate
        license: preemptAny
        criteria:
          - keys:
              - kind: age
              license: none
    - match:
        kind: allocated
        license: preemptInterruptible
        criteria:
          - keys:
              - kind: allocationBalance
                allocationBalance:
                  level: workspaceGroup
```

```

        license: preemptInterruptible
    - keys:
        - kind: allocationBalance
          allocationBalance:
            level: workspace
          license: preemptInterruptible
    - keys:
        - kind: priority
          license: preemptInterruptible
    - keys:
        - kind: age
          license: none
- match:
    kind: unallocated
    license: none
    criteria:
    - keys:
        - kind: priority
          license: preemptInterruptible
    - keys:
        - kind: age
          license: none

```

Partitions are evaluated in order (immediate, then allocated, then unallocated), and within a partition the criteria are the sort keys in order. A license marks where preemption rights are granted, so being ahead on workspace group balance can win a preemption, and being ahead on workspace balance can also win a preemption; being ahead on age only affects queue order.

Items To Confirm Before Publishing

- Exact semantics of `forgivenessHorizon` and `allowPreemptionVictimUnderQuota`, which are described conservatively above.
- Whether priority-based preemption in the unallocated partition applies only within the unallocated lane, as assumed above.
- The current GPU count on Holmes, if a concrete example of slot limits summing to cluster capacity should be included.
- A link to the scheduler presentation, and whether the two "Application of Allocations" slides should be embedded directly rather than reproduced as text.